lecture 10
amortized time

data structure	worst case for 1 operation	worst case for n operations	amortized time
A	$O(\sqrt{n})$		
B	$O(n)$	$O(n \log(n))$	$\frac{n\log(n)}{n} = O(\log(n))$

Data structure B is more scalable for big values of N

A: $n\sqrt{n}$
B: $n\log(n)$ } $n\log(n)$ is less than $n\sqrt{n}$

There are scenarios where you want a constant running time
- especially when code runs at special intervals

In these cases, data structure A is better since it is consistent

---

new tradeoff: time for 1 operation vs time for n sequence of operations

amortized time: average time for n operations

can have bad individual time, but average good time

$O(A(n))$ amortized time means n operations take $O(n\,A(n))$ total time

---

Growing Array

only 1 operation: appends value to end of array

[1, 2, 3, 4]
push 5

Challenge: you will run out of space and need to copy everything over to new memory, also free old memory

Naive design: array is always full and need to allocate and copy to new data

```
def push(array, val):
    new_data ← [request array, N+1 space]
    for i ∈ 0, 1, ..., array.N-1:
        new_data[i] = array.data[i]
    release array.data
    new_data[array.N] ← val
    array.data = new_data
    array.N += 1
```

Inserting value	1	2	3	4	5	6	7	8	9
n after insertion	1	2	3	4	5	6	7	8	9
size after insertion	1	2	3	4	5	6	7	8	9
time for insertion	$\le C$	$\le 2C$	$\le 3C$	$\le 4C$	$\le 5C$	$\le 6C$	$\le 7C$	$\le 8C$	$\le 9C$

Total: $\le 1C + 2C + 3C + \ldots + n \cdot C = C(1+2+3\ldots+n)$
$= C \cdot \frac{1}{2} n(n+1)$
$= O(n^2)$

amortized: $O(\frac{n^2}{n})$
$= O(n)$

divide total time for n operations by n to get amortized time

average case time: average over input distribution
amortized time: average of number of operations

only for algorithms
only for data structures

Adjustment: double array size when you request for more space

```
def push(arr, val):

    if arr.n == arr.s:
        new_data ← (request 2·(arr.N+1))

        for i ← 0,1,..., arr.N-1:
            new_data[i] ← arr.data[i]

        release old data
        arr.data ← new_data
        arr.S ← 2·(arr.N+1)
    arr.data[arr.N] ← val
    arr.N += 1
```

Inserting value	1	2	3	4	5	6	7	8	9
n after insertion	1	2	3	4	5	6	7	8	9
size after insertion	2	2	6	6	6	6	14	14	14
time for insertion	1c	c	3c	c	c	c	7c	c	c

Total: $c(1+1+3+1+1+1+7+1+1+\dots)$

2 6 14

each number doubles

Expensive operations are followed by cheap operations

$c(1\cdot 2 + 3\cdot 2 + 7\cdot 2)$

$\leq cn + cn = 2cn = O(n)$

Total: $O(n)$

Amortized: $O(1)$

Potential method: provide amortized time

Steps

- define potential function: Φ
  goes from data structure state to a real number

- theorem: Let S_0 be the initial state of the data structure and $S_1, S_2, \dots S_n$ be the state after n operations. Suppose you have a potential function Φ such that

1. $\Phi(S_i) \geq \Phi(S_0)$ potential state never drops after initial state

2. If d_i is the time taken for operation i. then:

$d_i + (\Phi(S_i) - \Phi(S_{i-1})) \leq f(n)$.

Then, the amortized time per operation is $O(f(n))$